

Customer Sentiment Analysis using MongoDB

11.1 NoSQL Database Creation and Data Insertion

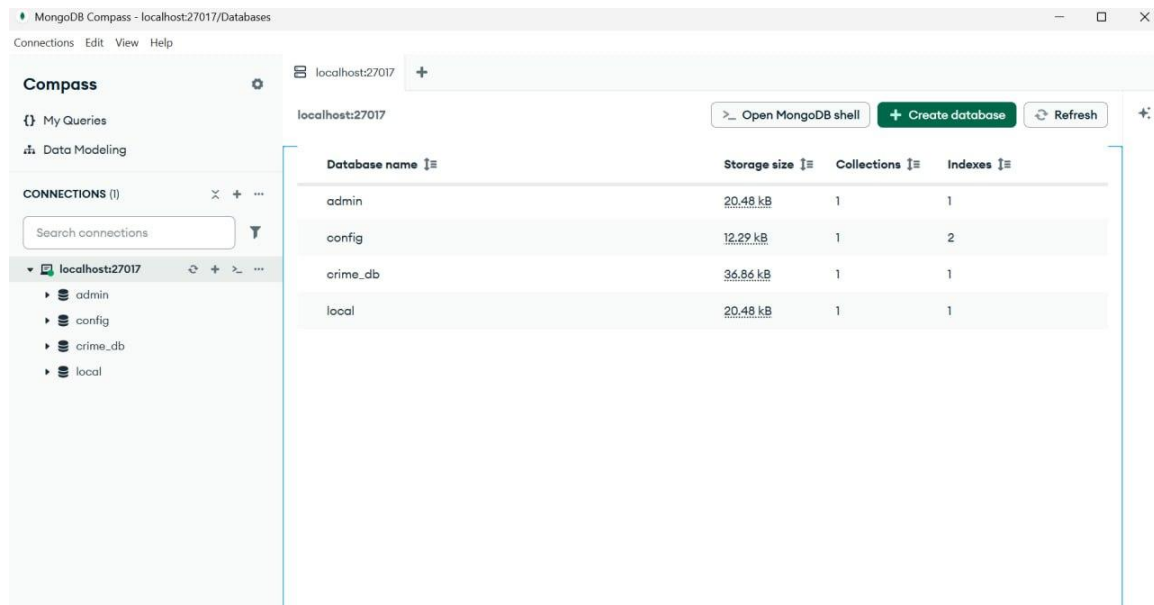


Figure 1: Create connection

The first step in the unstructured data analysis is to establish a connection to the MongoDB server and prepare the environment. We utilized **MongoDB Compass** to connect to the local MongoDB instance running on port 27017. As shown in the screenshot, the connection was successful.

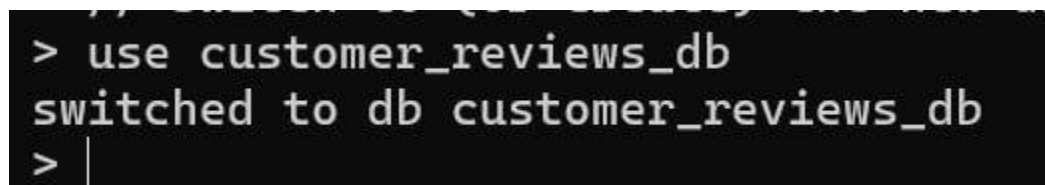


Figure 2: Switch to (or create) the new database

Create a new database named `customer_reviews_db` or switch to it if it already exists. This is where all our review data will be stored

List Of Products:-

1. Product 1 (ID: OFF-ST-10002): SAFCO Boltless Storage

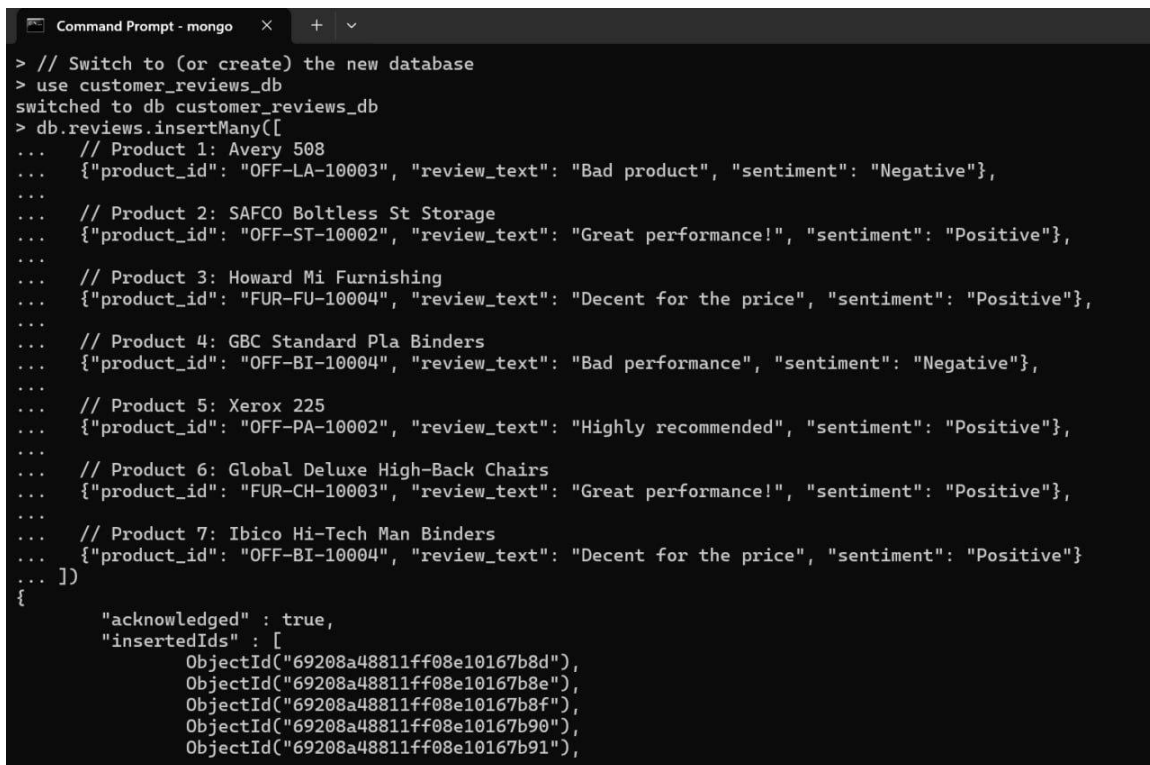
- Review 1: "Great performance!" (Positive)
- Review 2: "Bad product" (Negative)
- Review 3: "Works as expected" (Positive)

2. Product 2 (ID: TEC-PH-10004): GE 30524EE4 Phone

- Review 1: "Highly recommended" (Positive)
- Review 2: "Decent for the price." (Positive)

3. Product 3 (ID: FUR-FU-10004): Howard Miller 14-Furnishing

- Review 1: "Decent for the price." (Positive)
- Review 2: "Very bad quality, broke down" (Negative)



```
Command Prompt - mongo
> // Switch to (or create) the new database
> use customer_reviews_db
switched to db customer_reviews_db
> db.reviews.insertMany([
... // Product 1: Avery 508
... {"product_id": "OFF-LA-10003", "review_text": "Bad product", "sentiment": "Negative"},
...
... // Product 2: SAFCO Boltless St Storage
... {"product_id": "OFF-ST-10002", "review_text": "Great performance!", "sentiment": "Positive"},
...
... // Product 3: Howard Mi Furnishing
... {"product_id": "FUR-FU-10004", "review_text": "Decent for the price", "sentiment": "Positive"},
...
... // Product 4: GBC Standard Pla Binders
... {"product_id": "OFF-BI-10004", "review_text": "Bad performance", "sentiment": "Negative"},
...
... // Product 5: Xerox 225
... {"product_id": "OFF-PA-10002", "review_text": "Highly recommended", "sentiment": "Positive"},
...
... // Product 6: Global Deluxe High-Back Chairs
... {"product_id": "FUR-CH-10003", "review_text": "Great performance!", "sentiment": "Positive"},
...
... // Product 7: Ibico Hi-Tech Man Binders
... {"product_id": "OFF-BI-10004", "review_text": "Decent for the price", "sentiment": "Positive"}
... ])
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("69208a48811ff08e10167b8d"),
    ObjectId("69208a48811ff08e10167b8e"),
    ObjectId("69208a48811ff08e10167b8f"),
    ObjectId("69208a48811ff08e10167b90"),
    ObjectId("69208a48811ff08e10167b91"),
  ]
}
```

Figure 3: Insert review documents into the 'reviews' collection

load the sample customer reviews into a "Collection" named reviews. Each review is a "Document" containing the product ID, the review text, and the assigned sentiment (Positive/Negative).

```

> // Display all documents to verify insertion
> db.reviews.find()
{ "_id" : ObjectId("69208a4881ff08e10167b8d"), "product_id" : "OFF-LA-10003", "review_text" : "Bad product", "sentiment" : "Negative" }
{ "_id" : ObjectId("69208a4881ff08e10167b8e"), "product_id" : "OFF-ST-10002", "review_text" : "Great performance!", "sentiment" : "Positive" }
{ "_id" : ObjectId("69208a4881ff08e10167b8f"), "product_id" : "FUR-FU-10004", "review_text" : "Decent for the price", "sentiment" : "Positive" }
{ "_id" : ObjectId("69208a4881ff08e10167b90"), "product_id" : "OFF-BI-10004", "review_text" : "Bad performance", "sentiment" : "Negative" }
{ "_id" : ObjectId("69208a4881ff08e10167b91"), "product_id" : "OFF-PA-10002", "review_text" : "Highly recommended", "sentiment" : "Positive" }
{ "_id" : ObjectId("69208a4881ff08e10167b92"), "product_id" : "FUR-CH-10003", "review_text" : "Great performance!", "sentiment" : "Positive" }
{ "_id" : ObjectId("69208a4881ff08e10167b93"), "product_id" : "OFF-BI-10004", "review_text" : "Decent for the price", "sentiment" : "Positive" }
>

```

Figure 4: Display all documents to verify insertion

confirm that all the review documents we tried to insert in the previous step have been successfully saved into the database.

11.2 Querying for Positive and Negative Review Counts

```

}
}
> db.reviews.countDocuments({"sentiment": "Positive"})
5
>

```

Figure 5: Count all documents where sentiment is "Positive"

```

> db.reviews.countDocuments({"sentiment": "Negative"})
2
>

```

Figure 6: Count all documents where sentiment is "Negative"

Quick and simple way to get the total number of reviews that have the "Positive" sentiment tag. This command is repeated to count the "Negative" reviews.

```

> db.reviews.aggregate([ { $group: { _id: "$sentiment", count: { $sum: 1 } } } ])
{ "_id" : "Negative", "count" : 2 }
{ "_id" : "Positive", "count" : 5 }
>

```

Figure 7: Group documents by sentiment and count each group

This command uses the **Aggregation Pipeline** to group all documents by their sentiment (Positive or Negative) and then counts each group in a single operation. This is the preferred technique for large-scale projects.